Simulation-Based Bayesian Estimation of Macroeconomic Dynamic Models

Cameron Fen

December 2nd, 2022

Introduction

Motivation

- Estimating DSGE models crucial for many economic questions
- Limitation of the current method for estimation:

  1. Closed-form likelihood $\Rightarrow$ linearized solution method
  2. Slow filtering and MCMC procedure
     - Example MCMC estimation $\approx$ 10 million samples
     - No Filtering $\rightarrow$ faster/more accurate HANK estimation
  3. MSM estimation: Unrealistic Markov assumption
     - MSM approach: only few lags effect current data
     - My approach maintains state space relationship

Markov Assumption

My Contribution

- Sequential Neural Posterior Estimation (SNPE)
  - Technique borrowed from deep learning
- Resolving pain points, I will demonstrate SNPE has...
  - ...better Smets-Wouters (2007) posteriors than MCMC
    - SNPE requires 66x less iterations, but better posteriors
  - ...the ability to Bayesian estimate a Winberry (2018) heterogeneous agent model with all micro moments
    - 13x faster CPU-iteration compared to state-of-the-art
- SNPE compares favorably to filter & MCMC or MSM

Literature

The SNPE Algorithm

Bayes Rule

- Bayes Rule:

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$$

Variable	Name
θ	Parameters
X	Data
$P(\theta)$	Prior Distribution
$P(X\|\theta)$	Likelihood Distribution
$P(\theta\|X)$	Posterior Distribution

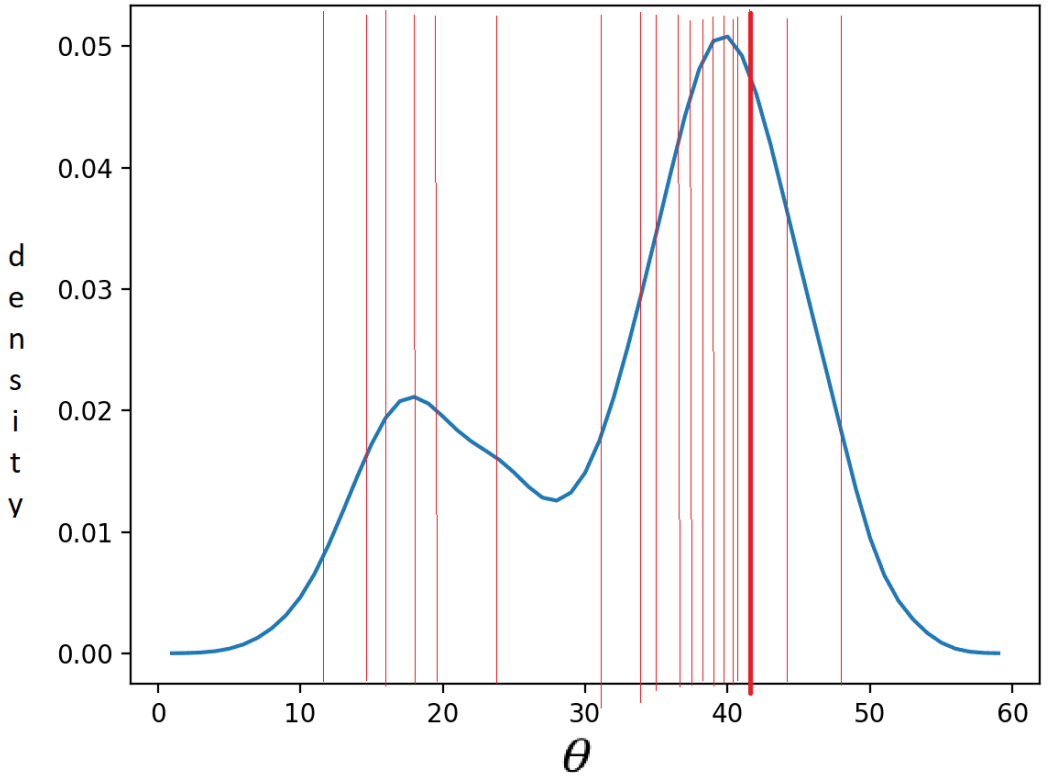
Example: $X_t = \rho X_{t-1} + \sigma\epsilon$

- X = data, $p(\theta)$ = prior distribution over θ
- Objective: Distribution of $\theta = (\rho, \sigma)$ given X
- **Step 1:** $\{X_i^*, \theta_i^*\}_{i=1}^I$ = simulated via Monte Carlo
  1. Sample $\theta_i^* = \{\rho_i^*, \sigma_i^*\}$ from the prior $p(\theta)$
  2. Sample $X_{it}^* = \rho_i^* X_{it-1}^* + \sigma_i^* \epsilon_t$
- **Step 2:** Estimate density $\mathbf{p}(\theta|\mathbf{X}^*)$
  1. Use conditional density estimator (CDE)

     Parametric Density Estimator
     - The CDE can be a normalizing flow, conditional mixture of Gaussians, or a GAN
- **Step 3:** Evaluate density conditioned on data $p(\theta|X^* = X)$

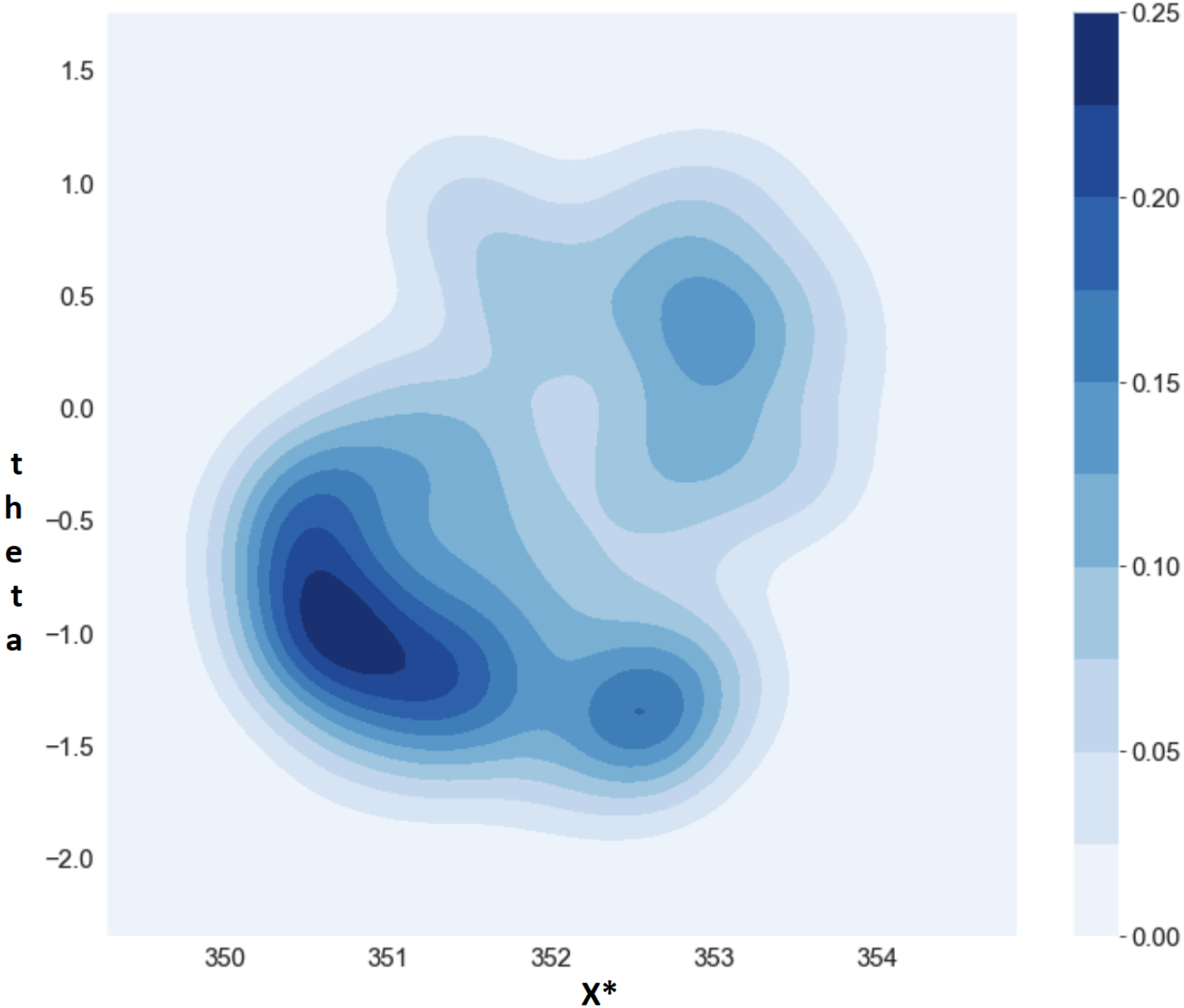
Conditional Density Estimation

Evaluating the Probability of a Point

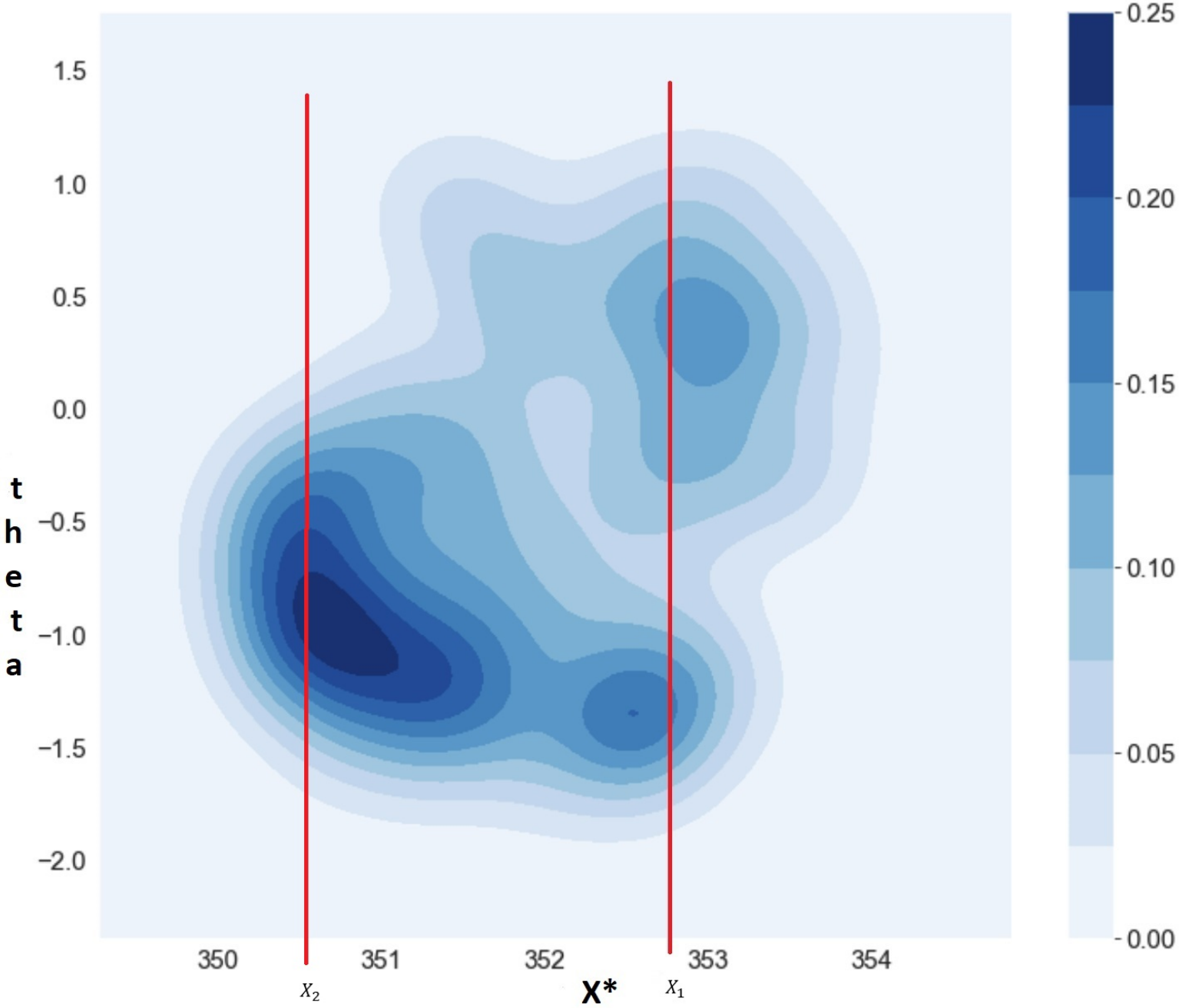


Back Big Picture Back SNPE

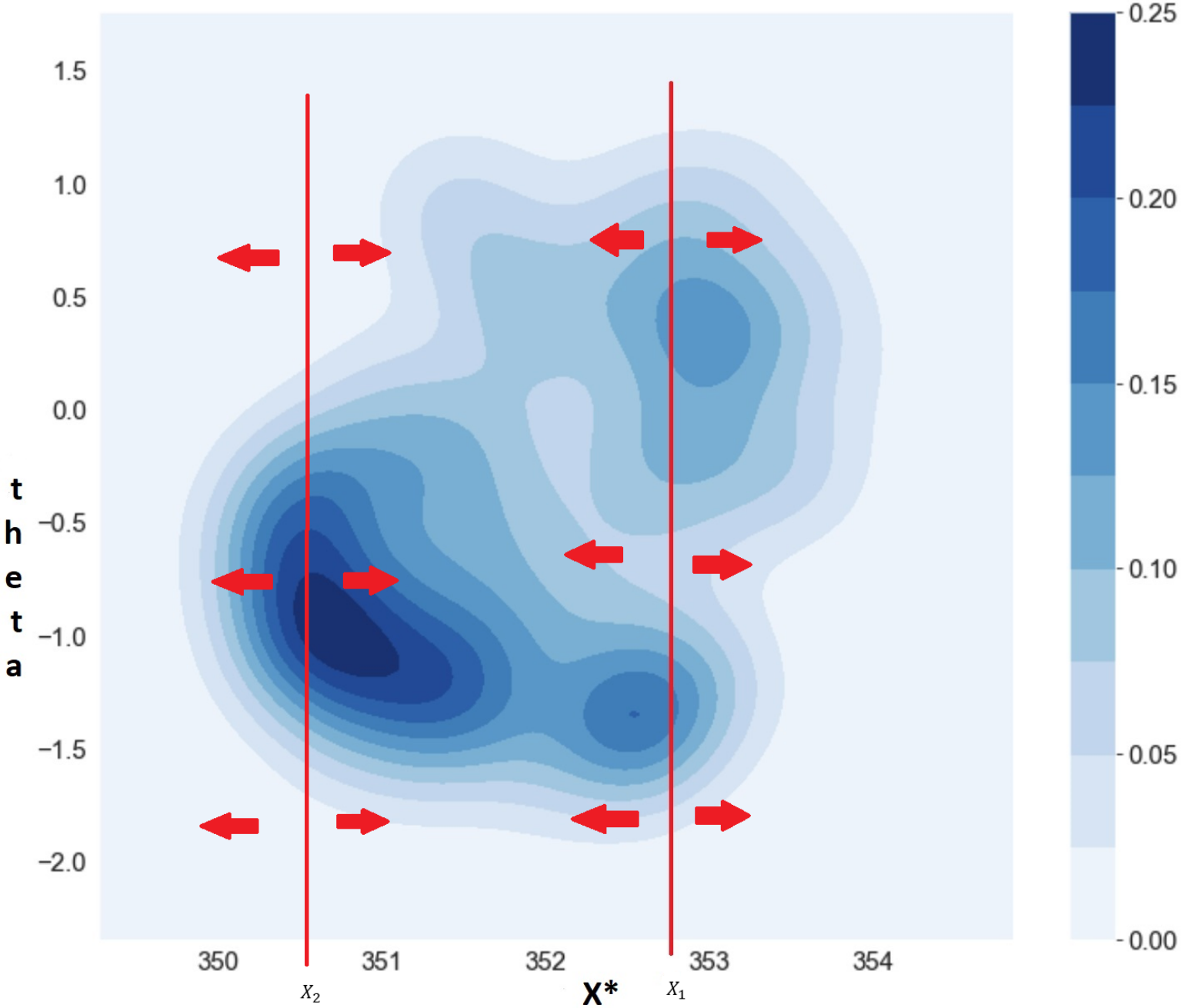
CDE with Parametric Density Estimators



Conditional Density Estimation



Conditional Density Estimation



One Example Conditional Mixture of Gaussians

- Equation: $\sum_i^n \alpha_i(x) * N(\mu_i(x), \Sigma_i(x))$
- The density is approximated by a mixture of Gaussian
- The weights and normal parameters change when x changes
- Likelihood: weighted average Gaussian likelihoods

Example: How to Perform Likelihood Density Estimation

- Estimating the likelihood of a point:
  1. Calculate likelihood of sample under each Gaussian
  2. Take weighted average sum of Gaussian likelihoods
- Maximize the likelihood:
  1. Calculate likelihood of a batch of points
  2. Using the likelihood gradient, optimize mixture model
     - The parameters: neural networks from x to α_i, μ_i, Σ_i

Other Density Estimators

- Can use normalizing flows or GANs as well
- Flow: conditional change of variable
  - Maps a known distribution to the distribution of interest
  - Optimize the parameters of the change of variables function
  - Utilize the density estimation property of GAN
  - Bayesian extension to (Kaji et. al. 2020) involves...
  - ...contrasting joint with the independent distribution

Example Redux: $X_t = \rho X_{t-1} + \sigma\epsilon$

- X = data, $p(\theta)$ = prior distribution over θ
- Objective: Distribution of $\theta = (\rho, \sigma)$ given X
- **Step 1:** $\{X_i^*, \theta_i^*\}_{i=1}^I$ = simulated via Monte Carlo
  1. Sample $\theta_i^* = \{\rho_i^*, \sigma_i^*\}$ from the prior $p(\theta)$
  2. Sample $X_{it}^* = \rho_i^* X_{it-1}^* + \sigma_i^* \epsilon_t$
- **Step 2:** Estimate density $\mathbf{p}(\theta|\mathbf{X}^*)$
  1. Use conditional density estimator
- **Step 3:** Evaluate density conditioned on data $p(\theta|X^* = X)$

Multi-Round Inference Importance Sampling

Example Redux: $X_t = \rho X_{t-1} + \sigma\epsilon$

- X = data, $p(\theta)$ = prior distribution over θ
- Objective: Distribution of $\theta = (\rho, \sigma)$ given X
- **Step 1:** $\{X_i^*, \theta_i^*\}_{i=1}^I$ = simulated via Monte Carlo
  1. Sample $\theta_i^* = \{\rho_i^*, \sigma_i^*\}$ from the prior $p(\theta)$
  2. Sample $X_{it}^* = \rho_i^* X_{it-1}^* + \sigma_i^* \epsilon_t$
- **Step 2:** Estimate density $\mathbf{p}(\theta|\mathbf{X}^*)$ ◀Previous Section
  1. Use conditional density estimator
- **Step 3:** Evaluate density conditioned on data $p(\theta|X^* = X)$

Multi-Round Inference Importance Sampling

Results

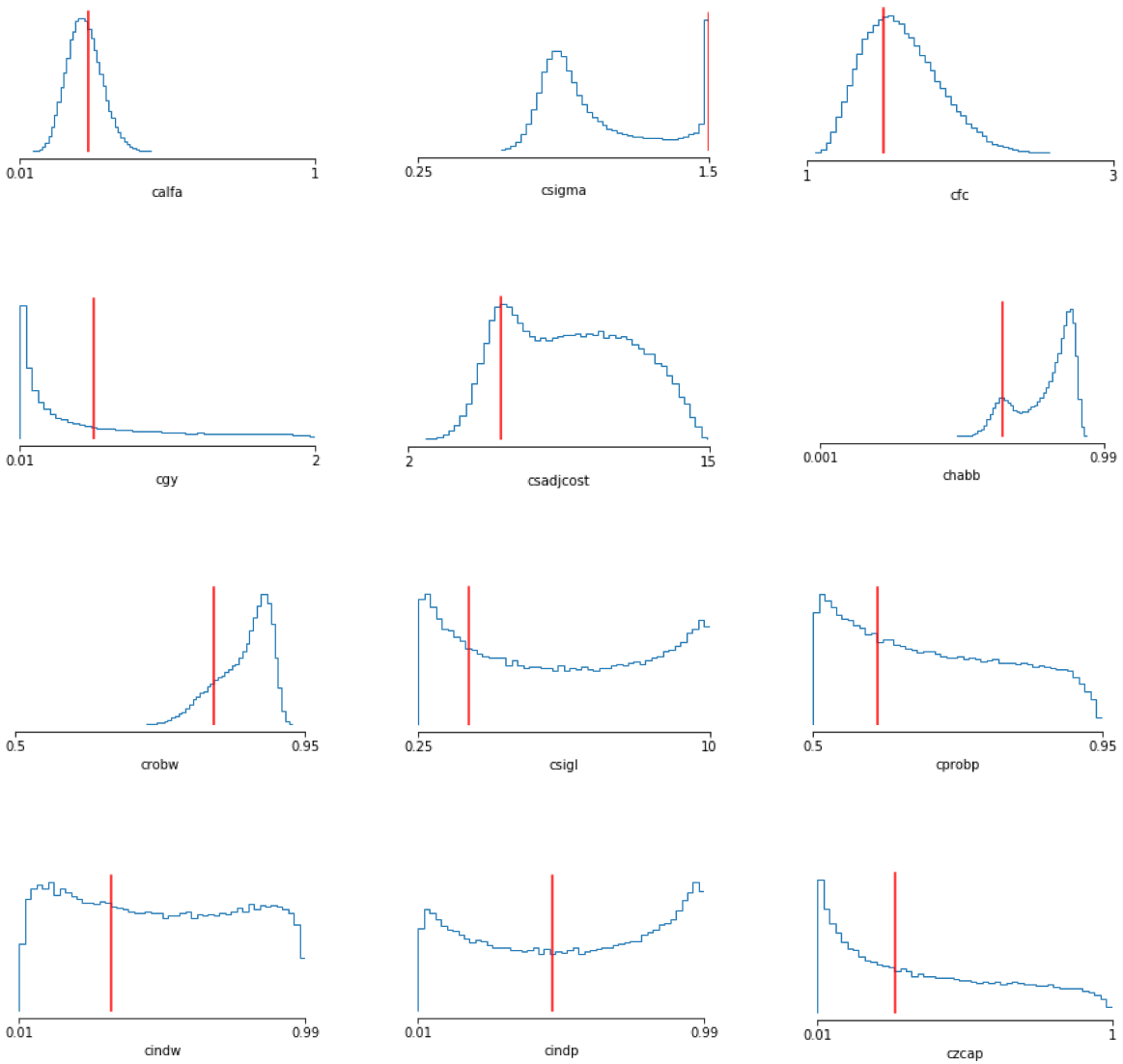
Ability to Estimate Different Models

Models	**SNPE**	**MCMC**	**Density Est**
RBC:	✓	✓	∼
Cash Flow:	✓	✗	∼
Lucas Tree:	✓	✗	∼
SW 2007:	✓	✓	∼
HANK:	✓	∼	∼
Bequests:	✓	✗	∼
KS 1998:	✓	∼	∼

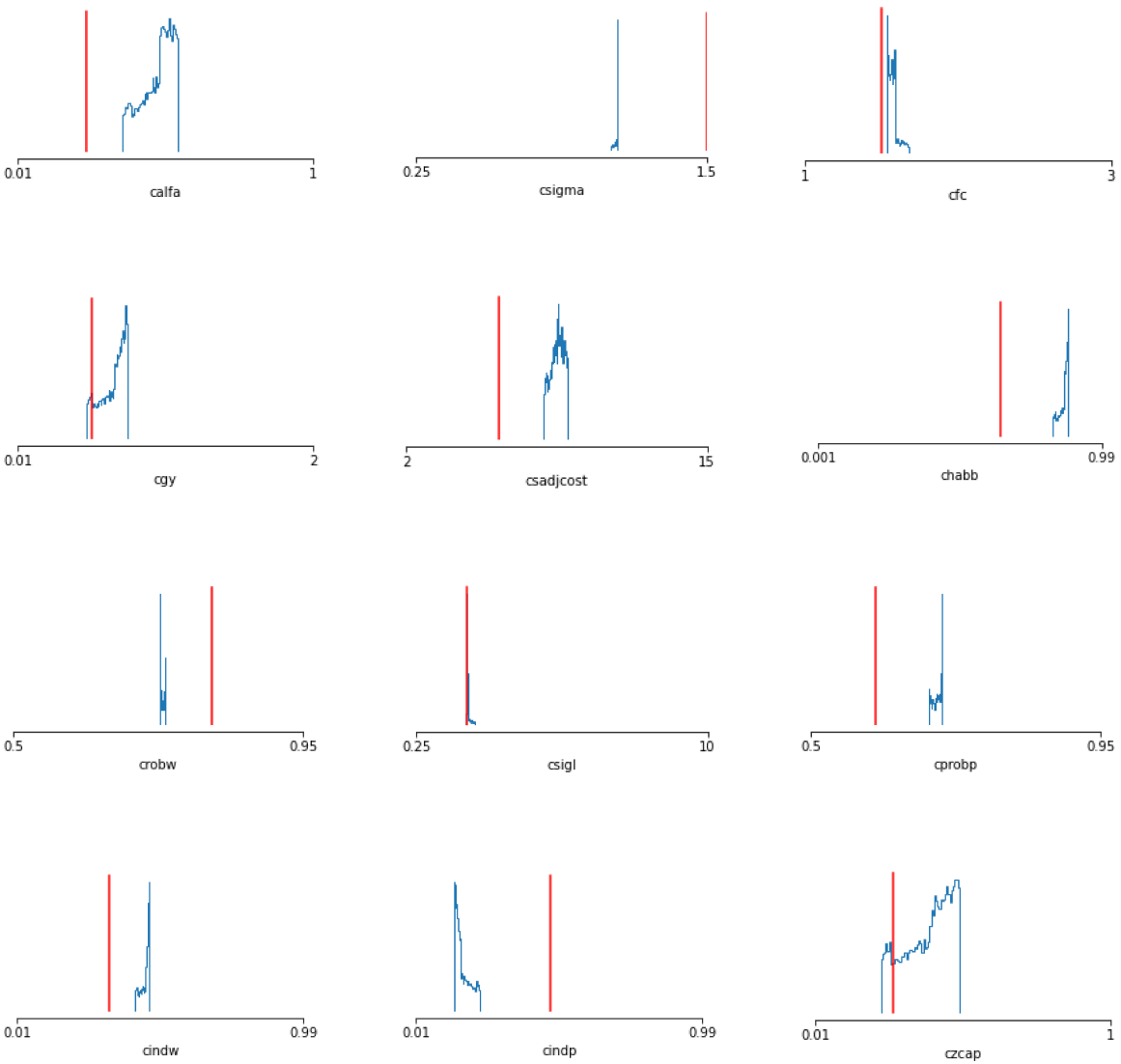
Case 1: Smets Wouters Model

- New Keynesian model based on Christiano et al. (2005)
- 36 parameters all estimated in a Bayesian fashion
- 7 observed variables and 7 autoregressive shocks

Smets-Wouters SNPE after 500K iterations: Params 1-12



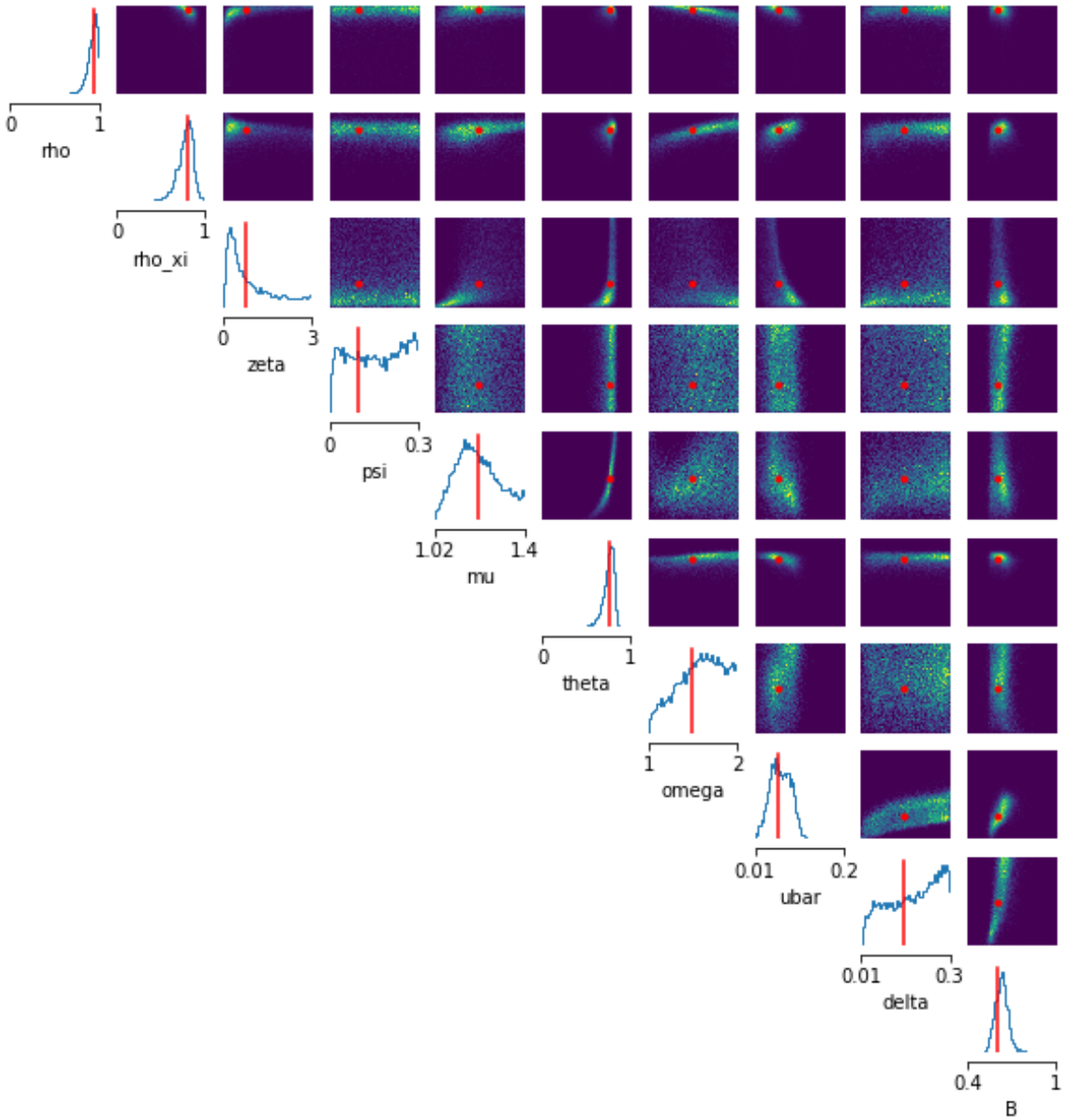
Smets-Wouters MCMC after 10M iterations: Params 1-12



HANK Model

- From Alisdair McKay
- Wealth is the main distributional state
- Production function linear in productivity and labor
- Standard New Keynesian backbone (Dixit-Stiglitz aggregator, Calvo pricing etc.)
- Labor with search, government with bonds

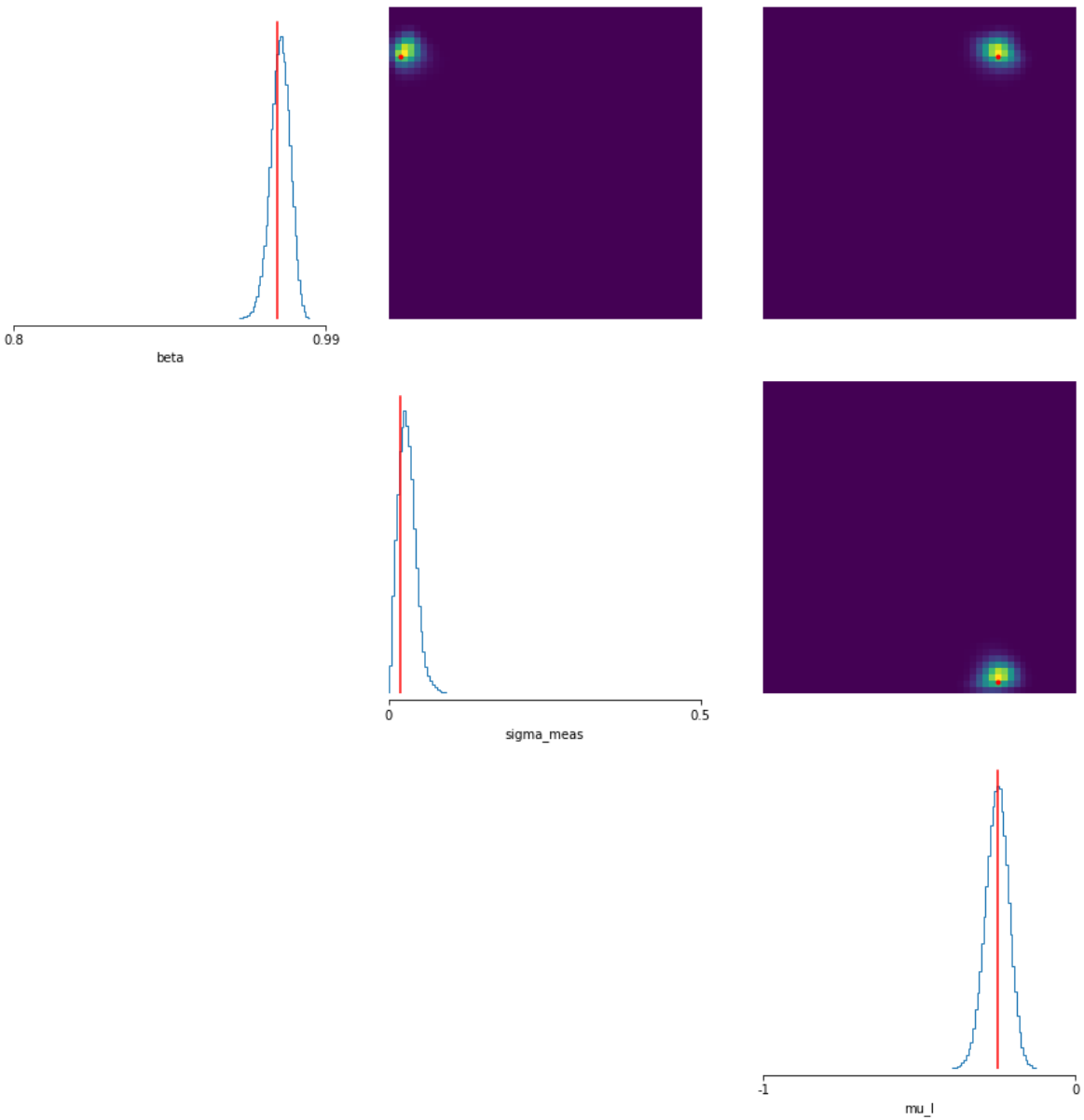
10 Parameter HANK Model



Case 2: Bayesian Krusell Smith (1998)

- Following Liu and Plagborg-Moller 2021 (LPM 2021)...
- ...I estimate KS with all micro moments
- KS has heterogeneous agents who differ in assets
- KS also has aggregate state variables
- Solved via Winberry (2018)
- LPM 2021 take 6.5 min. per CPU iteration...
- ...my approach takes 30 seconds

Krusell-Smith via Winberry: 10k Samples



Comparison to Previous Work

- Kristensen and Shin 2012 use a KDE in similar algorithm
- Likewise Kaji, Manresa, Pouliot 2022 with their GAN
  - Both perform only 'MLE' with Markov Assumption
- KDE/GAN $\rightarrow$ my CDEs $\Rightarrow$ faster true Bayesian inference
  - SNPE Bayesian inference is more accurate than MCMC

Conclusion

- SNPE: simulation-based Bayesian/MLE estimation
- Advantage over filtering + MCMC:
  - Better fit, faster, and works for all solution methods
- Advantage over other MSM and like methods:
  - Likelihood, efficiency, and state space assumption
- All advantages theoretically or empirically demonstrated

Thank You

Email: camfen@umich.edu
Twitter: @cameronfen1
Website: cameronfen.github.io

Appendix

Literature

- Many models are non-smooth:
  - **s-S Models**: Arrow et al. 1951, Caplin and Spulber, 1987, Bertola and Caballero 1990, House and Leahy 2004, Caplin and Leahy 2006, Kahn and Thomas 2008, Kaplin et al. 2020, Bardóczy 2022, etc
  - **Structural Finance**: He, Whited, and Guo 2021, Terry, Whited, and Zakolyukina, 2021, Taylor 2012, Whited and Guo 2006 etc.
- Non-smooth models: one can simulation-based estimate:
  - **Method of Simulated Moments**: McFadden 1989, Pakes and Pollard 1989, Duffie and Singleton 1990, etc.
  - **Machine Learning**: Kaji, Manresa, and Pouliot 2020

Back

Importance Sampling

- In practice one often samples and re-estimates the posterior multiple times
- In the first round one samples $\theta \sim P(\theta)$
- But in later rounds one wants the proposal distribution for θ to be as close to the posterior as possible
- Thus in later rounds of estimation one usually samples $\theta \sim P'(\theta|X = X)$, the estimate of the posterior obtained in the preceding round
- In order to correct for sampling from the "wrong" distribution, instead of estimating just the normalizing flow density estimator $f_\phi(\theta|x)$, one estimates the density estimator along with an importance weighting factor, $f_\phi(\theta|x)\dfrac{p^{(i-1)}(\theta)}{p(\theta)}$ on samples (X, θ).

Back

Why would Embeddings Perform Better than PCA

- The intuition for why end-to-end dimensional reduction outperforms PCA:
  1. Given a feature extraction approach, one extracts features to optimize an auxilliary loss
     - For example PCA optimizes a different loss function then our log likelihood in the flow
  2. Since this loss is different, the features extracted will not contain the optimal information to satisfy our loss function
  3. In an end-to-end model, the feature extraction step in, for example, my flow, is trained along with the density estimator so that the optimal features are learned to maximize log likelihood

Back

Why would Embeddings Perform Better (cont)

- Replacing feature extraction approaches with end-to-end neural network learning is partially responsible for much of the improvement like Alexnet in the Imagenet contest, and deep reinforcement learning for self driving cars
- A common theme in early deep learning to get out-performance, was to replace feature extraction approaches with end-to-end learning

Back

Bijectors: Dimensionality Greater than 2

- Using the same logic, one can find z_i from θ_i
- Assuming one knows $z_1 \ldots z_{i-1}$, then...
- ...$a_i(.)$ and $b_i(.)$ are only functions of $z_1 \ldots z_{i-1}$:
  - Thus, a_i and b_i are known and one can solve for z_i

Back

The Entire Algorithm: SNPE

Algorithm 1: SNPE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, flow $f_\phi(x|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)(\theta)} = p(\theta)$, data set D $= \{\}$;

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $f_\phi(x, \theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

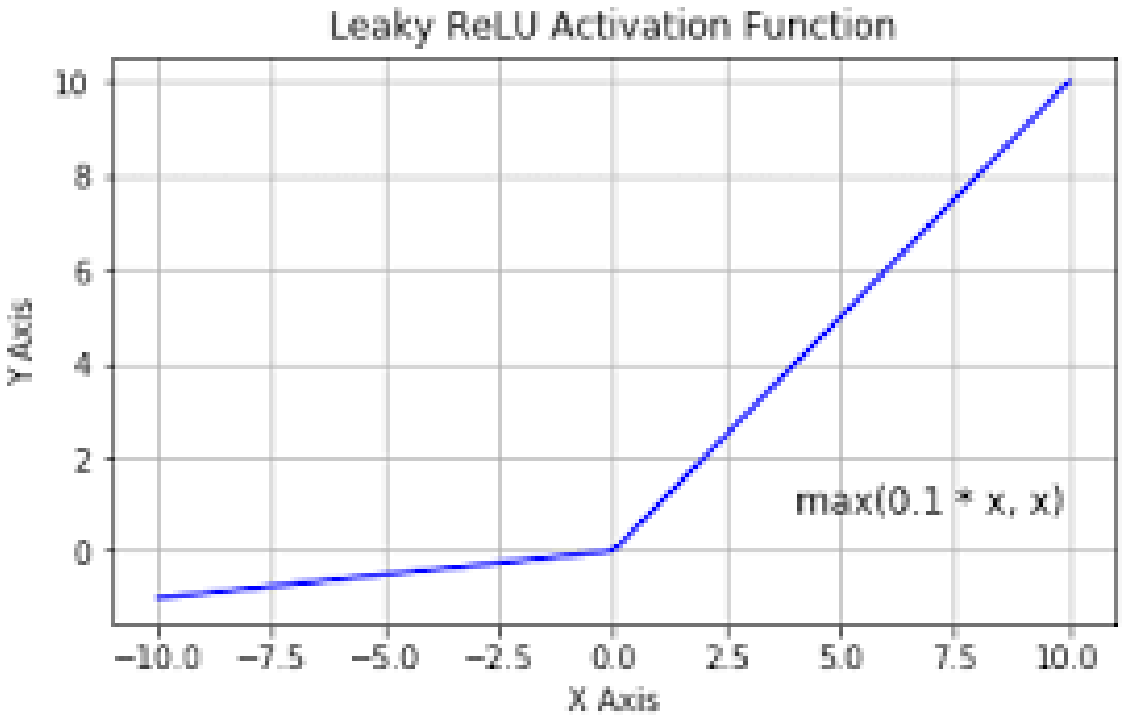
    Train $f_\phi(\theta|x)\dfrac{p^{(i-1)}(\theta)}{p(\theta)}$ on $\{x^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)(\theta)} = f_\phi(\theta|x_0)$;

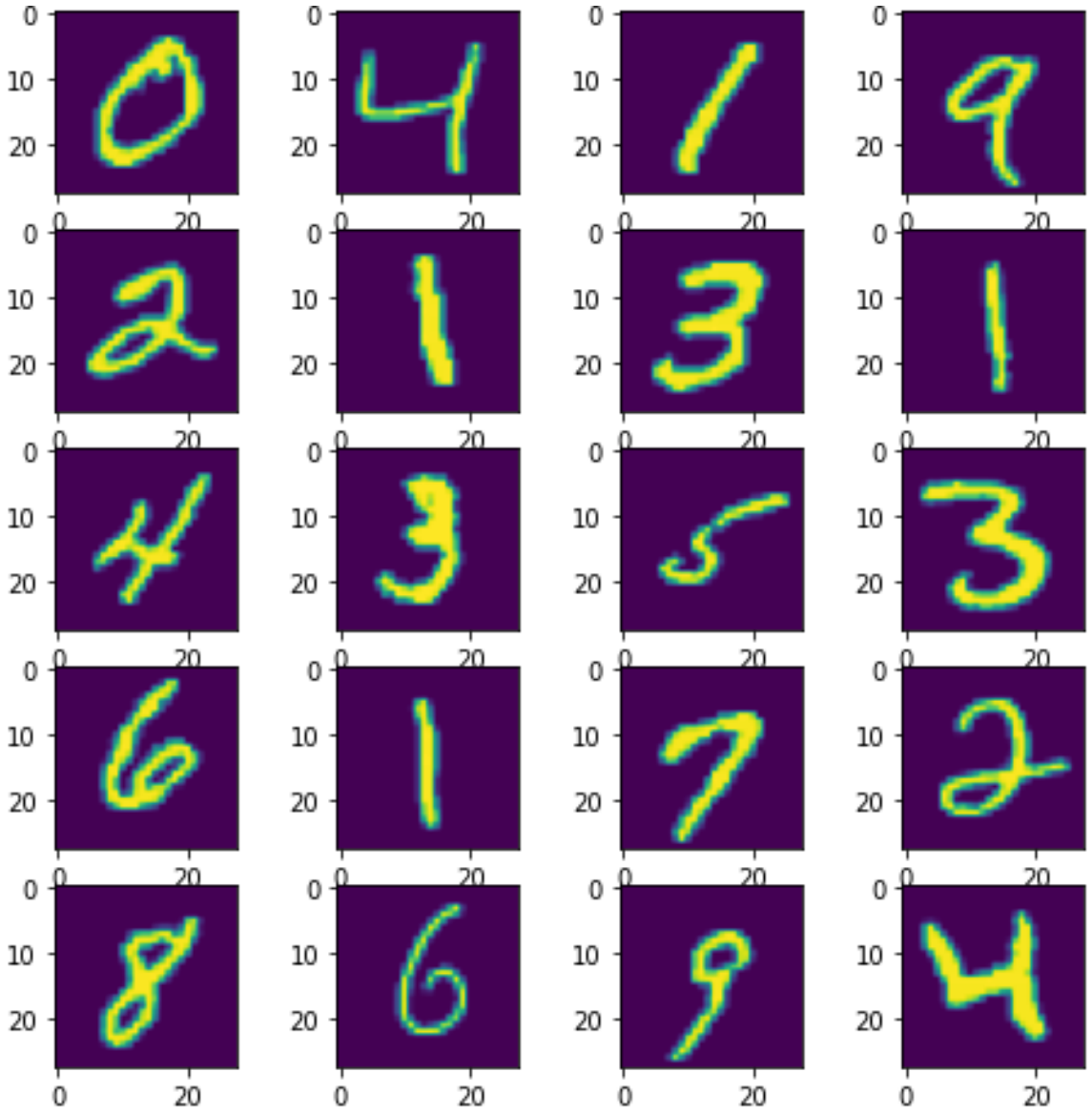
end

Phi: The Nonlinearity



Back

Normalizing Flow Example



Flow Universal Approximator Proof

Theorem (Flow Universal Approximator)

1. The Normalizing Flow is a Universal Approximator of Monotonic Functions (Sketch)

- First I show a step function can universally approximate any monotonic function using an argument similar to universal approximators of simple functions in measure theory.
- Then I show that a logistic regression can arbitrarily approximate any step function.
- Since $\alpha(.)$ and $\beta(.)$ are feed forward neural networks which are universal approximators of continuous functions, a neural autoregressive flow can arbitrarily approximate any logistic regression.
- Thus flows universally approximate monotonic functions

Flow Universal Approximator Proof

Theorem (Monotonic Functions and Distribution Mapping)

2. Monotonic Functions Can Universally Map Any Continuous Distribution to Any Other (Sketch)

- This proof implies that any distribution can be transformed to any other distribution via a monotonic function that is a change in variables. The proof resembles Sklar's theorem for copulas.
- First one can transform any distribution to a uniform distribution via a monotonic function by recognizing that the CDF of any distribution is exactly that transform.
- Likewise the inverse CDF allows one to transform to a uniform distribution from any other distribution via a monotonic change in variable
- Since one can map any distribution to a uniform and a uniform to any distribution, all distributions can be

Change of Variable Proof

Theorem (Radon Nikodym Theorem for Normalizing Flows)

1. *Normalizing Flow is a Valid Change of Variable (Sketch)*
   - The neural autoregressive flow as set up is a bijection, if one sets the parameters of the $\alpha(.)$ and $\beta(.)$ neural networks correctly.
   - As long as all the parameters are bounded, the neural autoregressive flow is Lipschitz, which implies that the input and output are absolutely continuous with respect to one another.
   - Thus, one can apply the Radon-Nikodym theorem on the change in measure in both directions on the base and target distribution.
   - This means the change of variable formula in the forward direction is: $P(y) = P(z)|det(df/dz)|^{-1}$, as discussed above.

The Jacobian

1. Given a monotonic normalizing flow transformation $f(z)$
2. The CDF of $f(.)$ is $P_f(\theta) = P(f(z) < \theta)$
3. This equals: $P(z < f^{-1}(\theta))$
4. Calculating the pdf by taking derivative: $\frac{d}{dy}P(z < f^{-1}(\theta))$
5. Using the chain rule: $p_f(\theta) = p(z = f^{-1}(y))|det(\frac{df^{-1}}{d\theta})|$
6. By inverse function theorem and determinant properties:

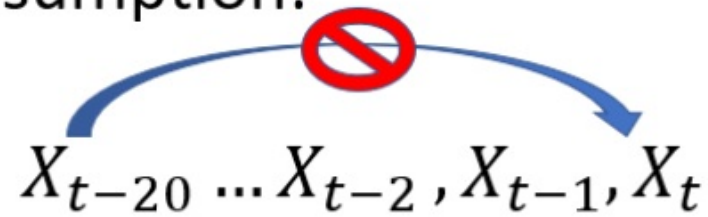
$$|det(\frac{df^{-1}}{d\theta})| = |det(\frac{dz}{df})| = |det(\frac{df}{dz})|^{-1}$$

- Intuitively, the Jacobian makes the integral sum to 1

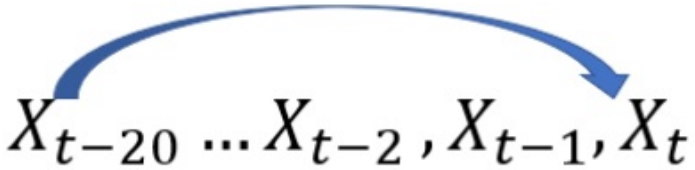
Back

MSM/GAN Markov Assumption vs SNPE State Space Assumption

- MSM, GAN Markov Assumption:



- The State Space Assumption:



Back

Model of Firm Capital Formation

- From Toni Whited and Emil Lakkis
- Same as the RBC model, except profits after investment is the utility function

$$U(K_t, I_t, Z_t) = \sum_t \beta^t * (\pi(K_t, Z_t) - I_t) \tag{1}$$

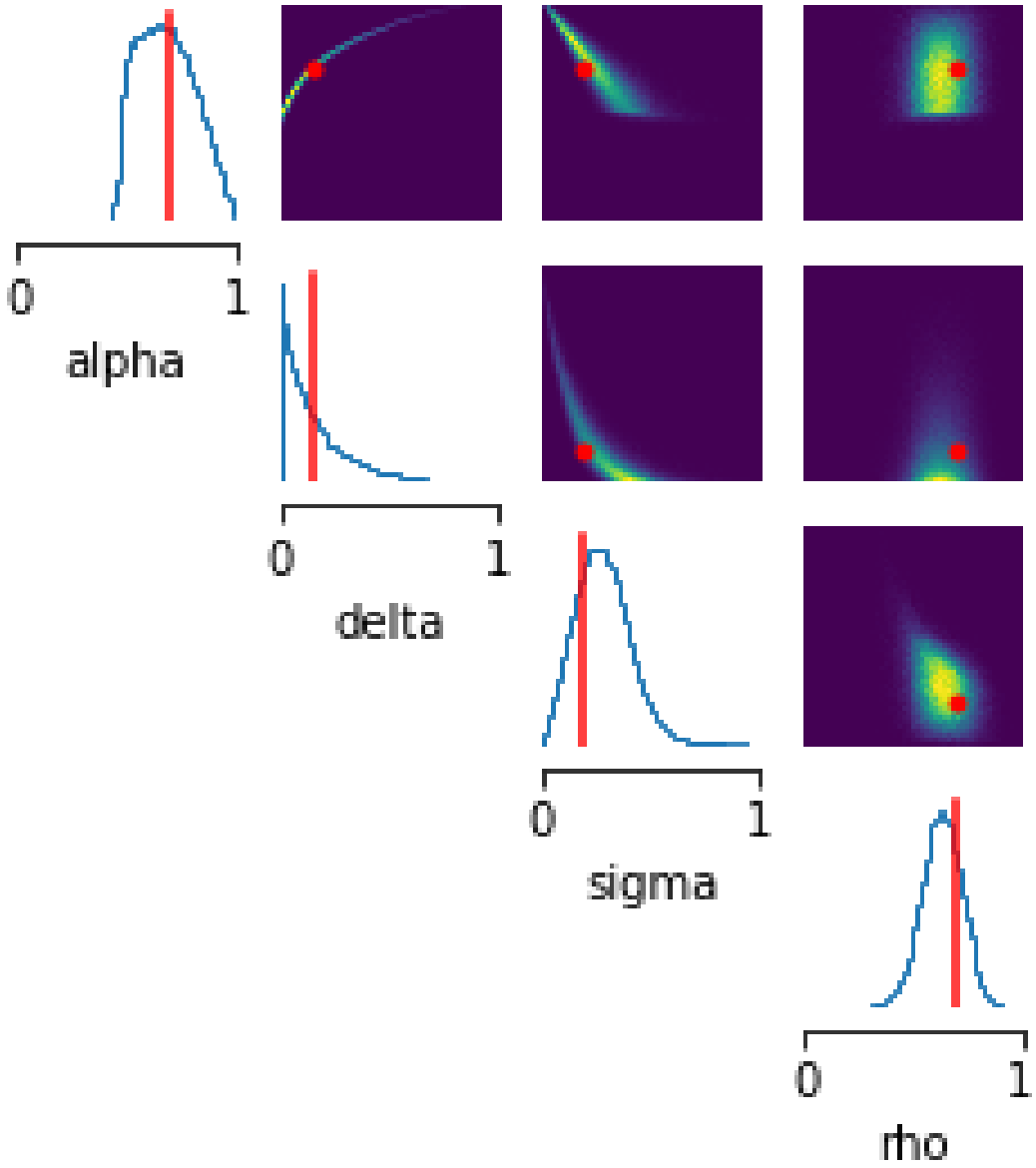
$$K_t = (1 - \delta)K_{t-1} + I_t \tag{2}$$

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{3}$$

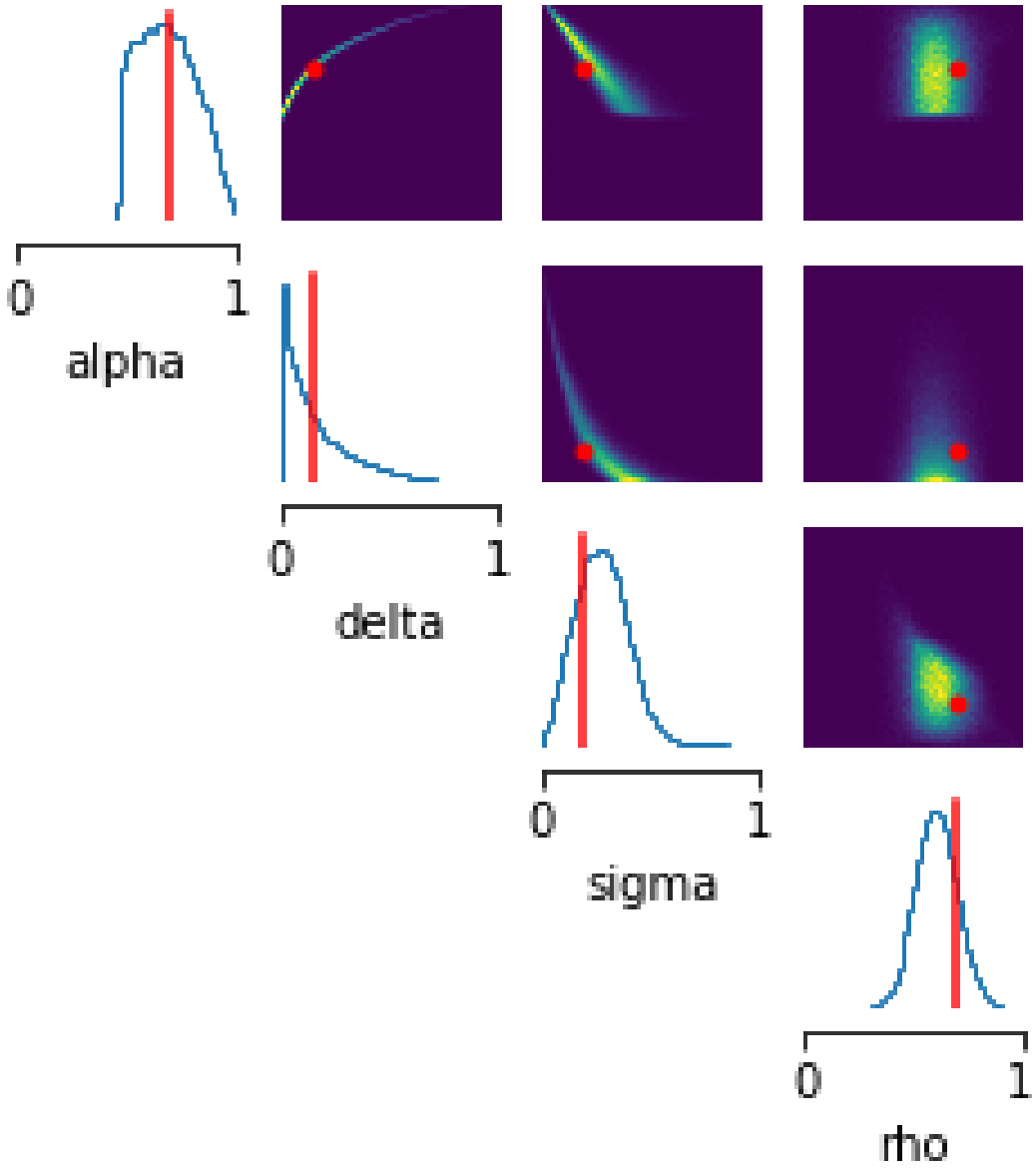
$$\pi(K_t, Z_t) = Z_t K_t^{\alpha} \tag{4}$$

$$\tag{5}$$

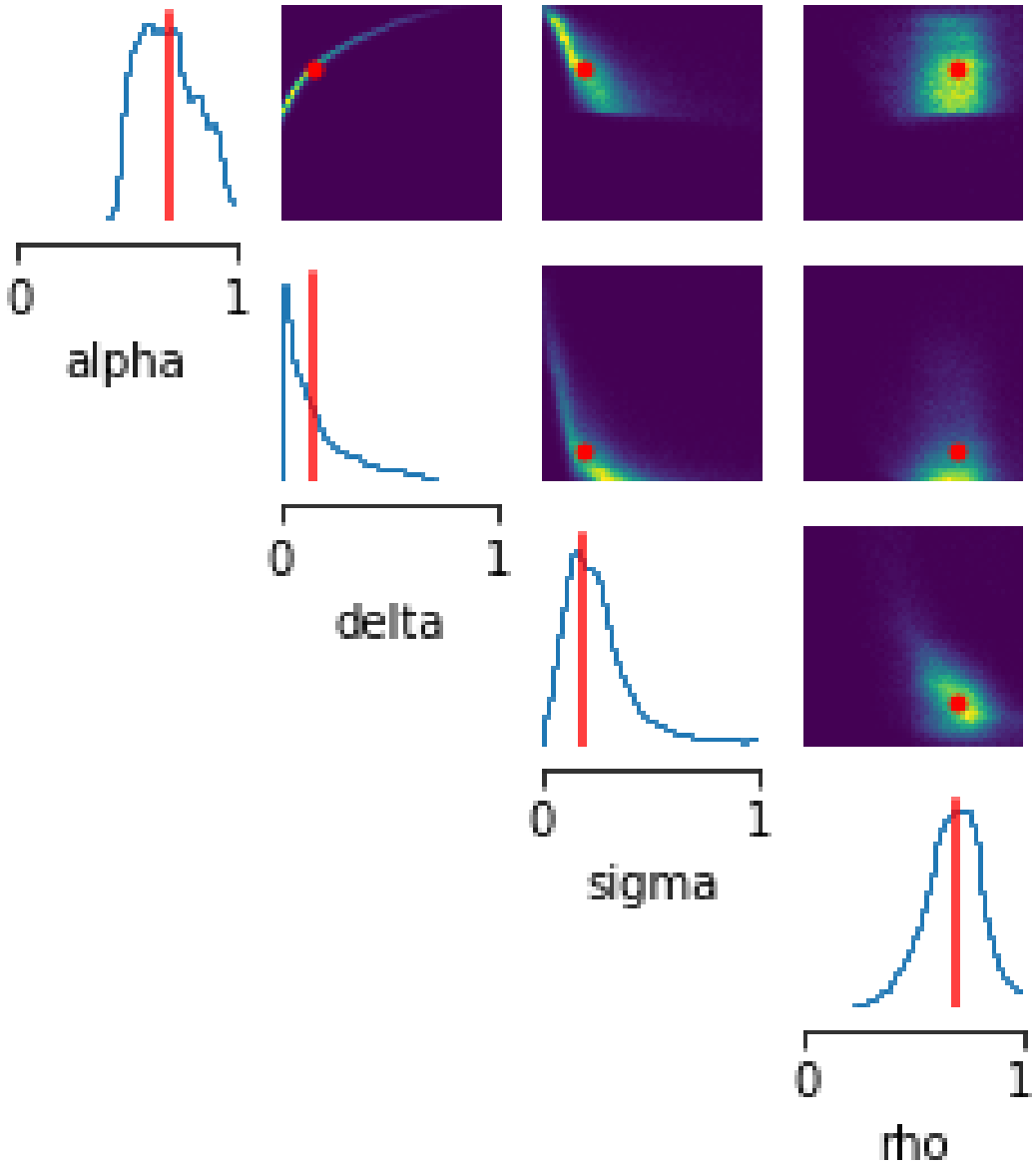
Value Function Iteration with Dense Net Embedding



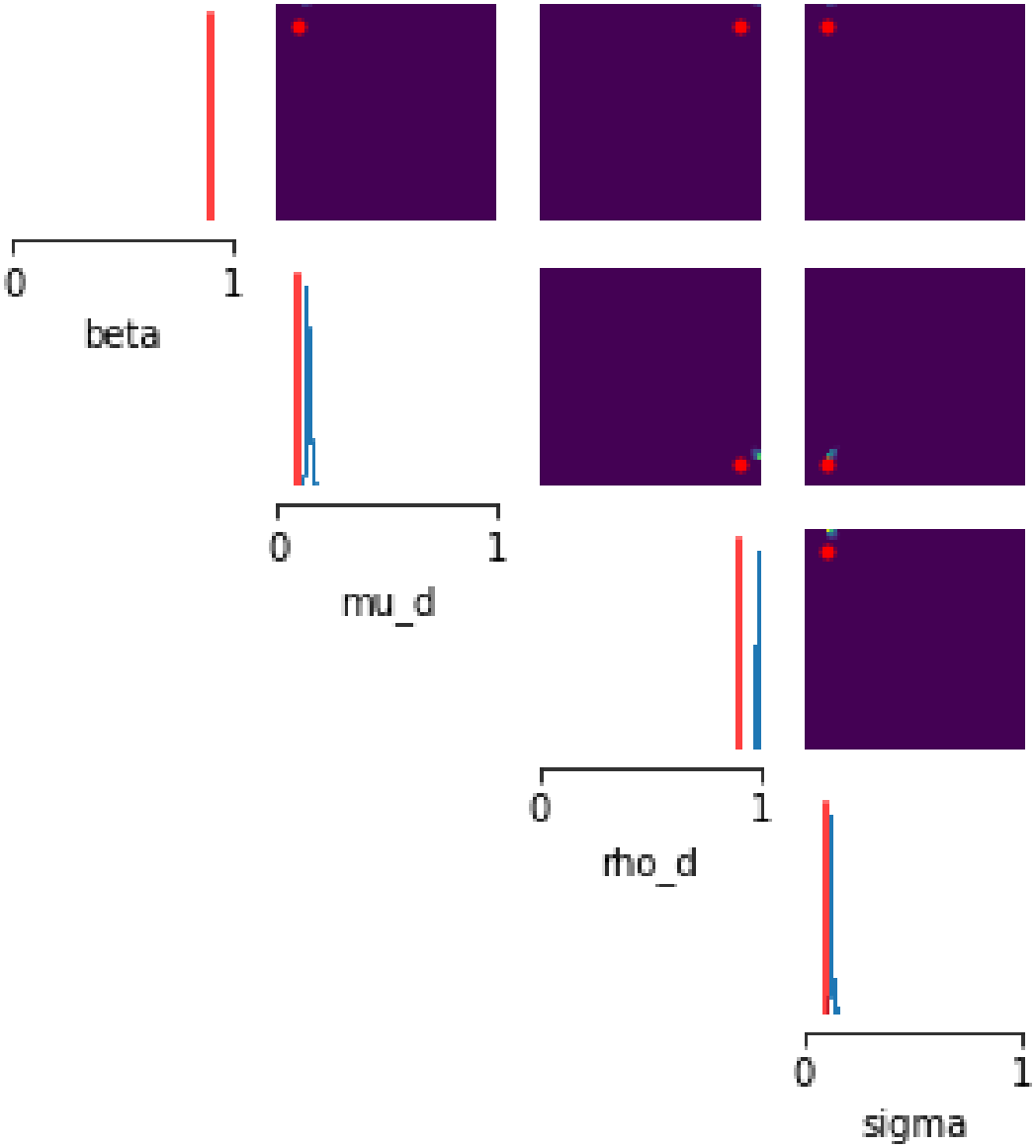
Value Function Iteration with RNN Embedding



Value Function Iteration with GAN Density Estimator



Lucas Tree Projection Model



RBC Model Description

- From Alisdair McKay: https://alisdairmckay.com/Notes/HetAgents/index.html
- Standard CRRA utility:

$$u(C_t) = E \sum_t \beta^t \frac{C_t^{1-\gamma}}{1-\gamma} \tag{6}$$

- Cobb-Douglas production, standard law of motion, and productivity AR in logs

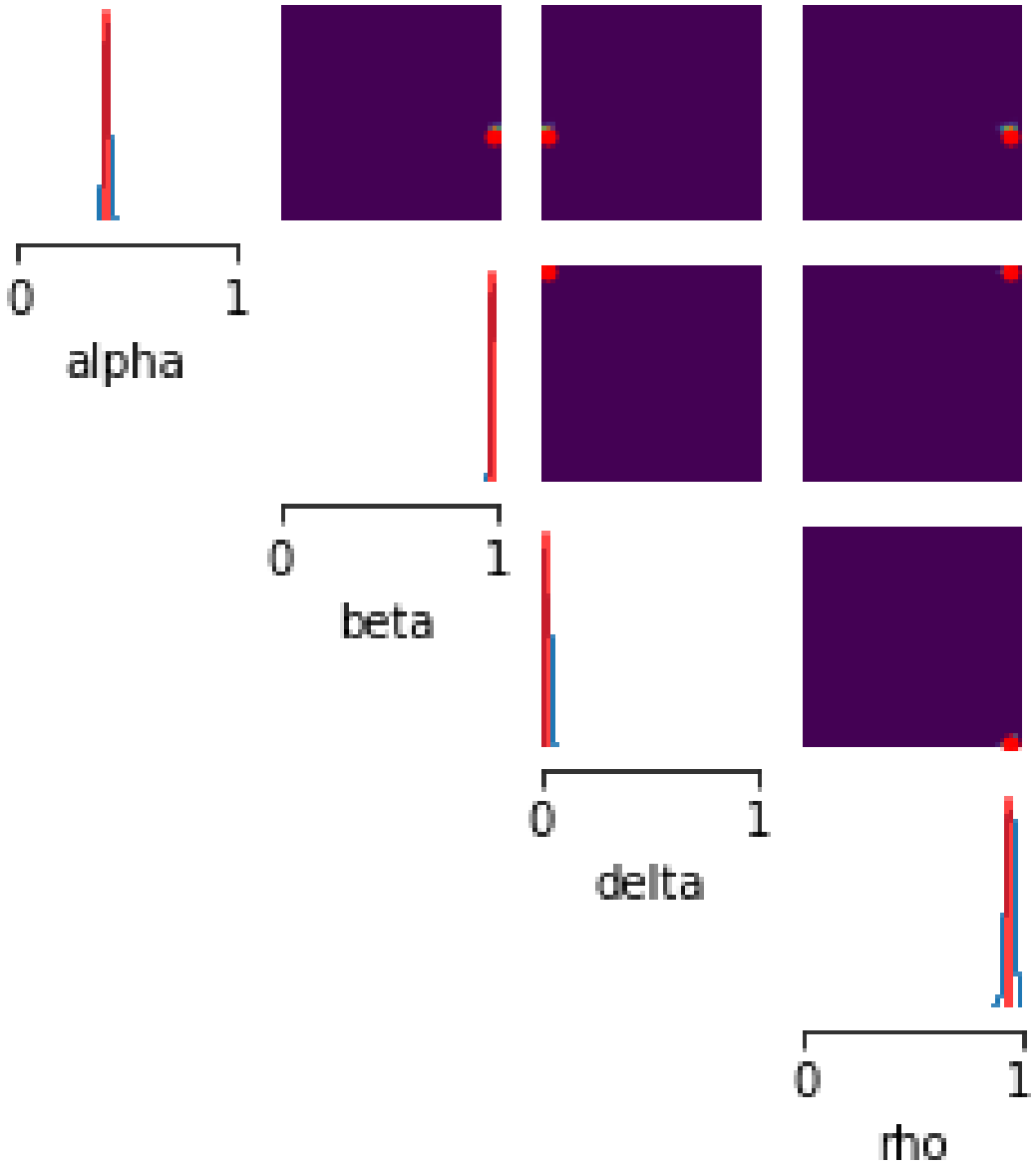
$$Y_t = Z_t K_t^{\alpha} \bar{L}^{1-\alpha} \tag{7}$$

$$K_t = (1-\delta)K_{t-1} + Y_t - C_t \tag{8}$$

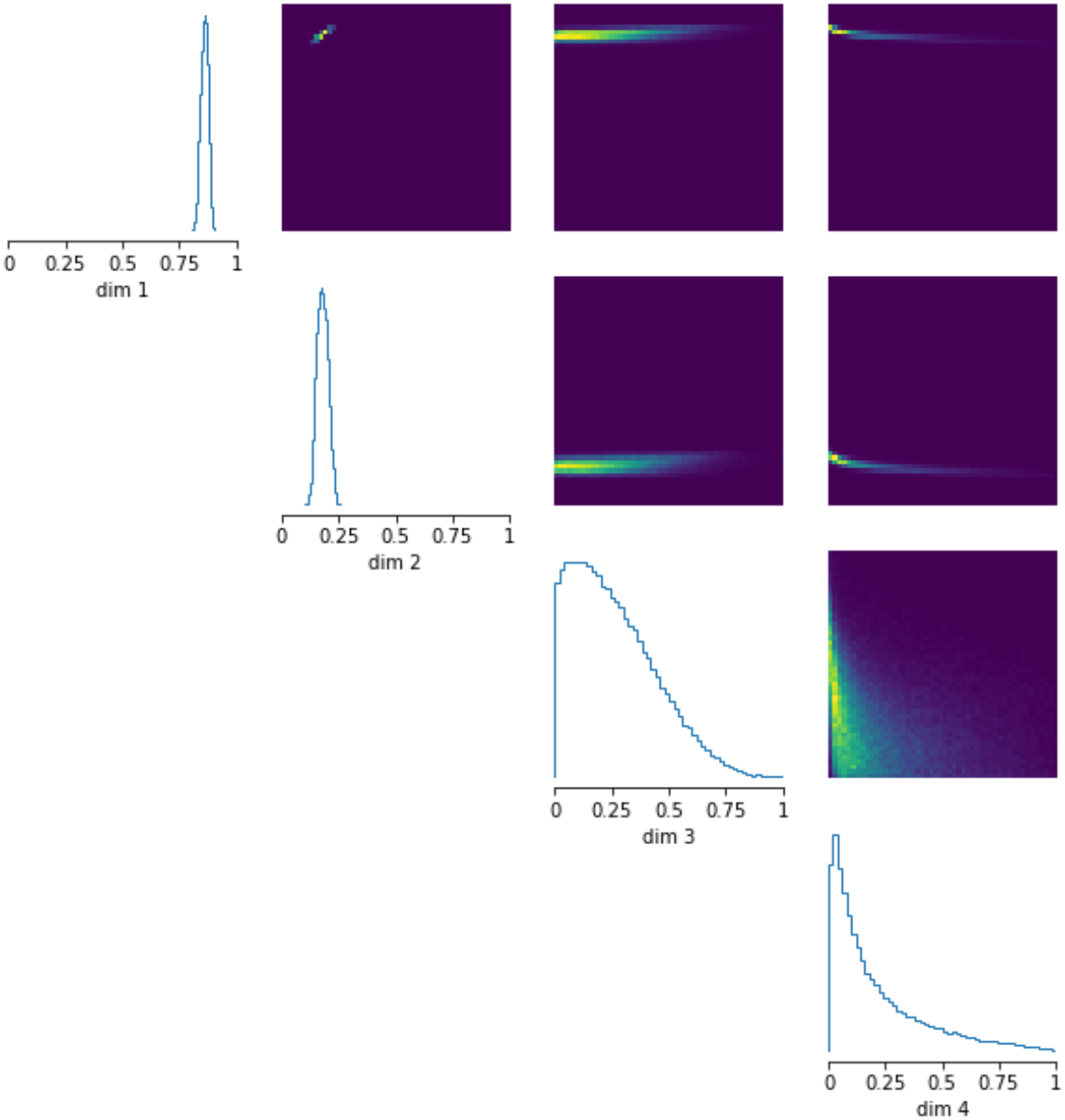
$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{9}$$

$$\tag{10}$$

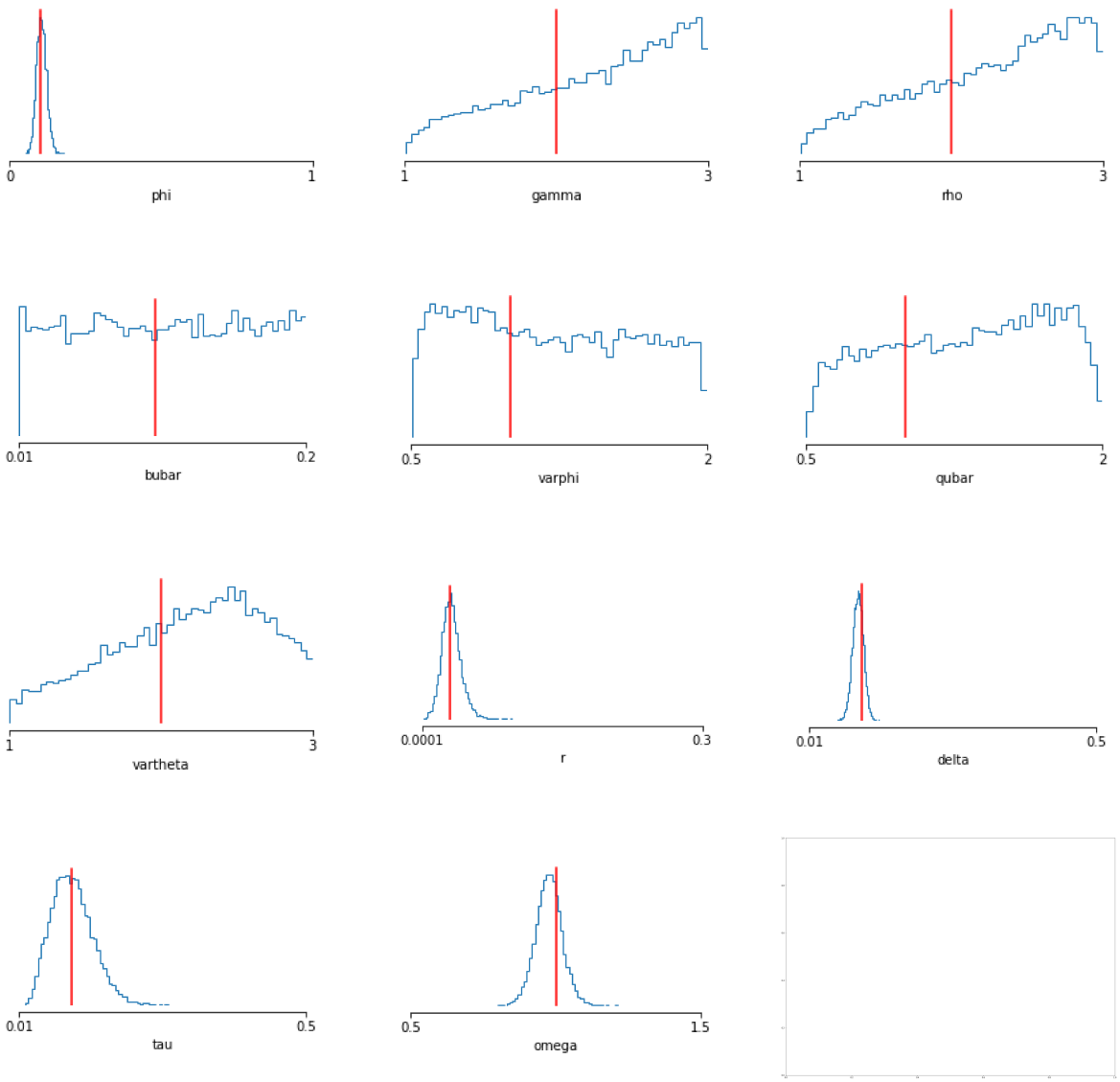
Simulated Data RBC



Real Data RBC



Case 2: Bequests Value Function Iteration Model

- This is a bequests model with a liquid and illiquid asset
  - Liquid assets can be used in consumption
  - Illiquid asset decays over time but provides a flow benefit
- Policy function kink where liquid to illiquid asset switch
- Solved via value function iteration

11 Parameter Value Function Iteration Model